Reproduction Report: PP#20

Sherman: A Write-Optimized Distributed B+Tree Index on Disaggregated Memory

CS6204 Spring 2026 | Chen-Wei (Wilson) Chang

1. Overview

Sherman is a write-optimized distributed B+Tree index designed for disaggregated memory architectures. The key contribution is Hardware-Offloaded Concurrent Locks (HOCL), which leverages the on-chip SRAM of RDMA NICs to implement a lock table that eliminates the need for remote RDMA round-trips when acquiring locks. The paper reports 24.7x higher throughput in write-only and 23.6x higher throughput in write-intensive skewed workloads compared to FG+, the state-of-the-art baseline.

This report documents the reproduction of Sherman's core performance experiments (Figures 10 and 11 from the paper), the hardware and software setup challenges encountered on CloudLab, and a comparison of our measured results against the paper's reported numbers.

2. Experimental Setup

2.1 Setup Comparison

Due to resource constraints on CloudLab, we used a reduced 2-node setup (1 memory server + 1 compute server, colocated on each node) instead of the paper's 16-node configuration. Benchmarks were collected on c6525-100g nodes (Utah cluster).

Component	Paper Configuration	Our Configuration
Nodes	8 MSs + 8 CSs (16 total)	2x CloudLab c6525-100g (Utah)
CPU	2× Intel Xeon E5-2650 v4 @ 2.2GHz	24-core AMD EPYC 7402P @ 2.80GHz
RAM	128GB DRAM per node	128GB ECC (8x 16GB 3200MT/s RDIMMs)
Disk	Not specified	Two 1.6TB NVMe SSD (PCIe v4.0)
NIC (control)	-	Mellanox ConnectX-5 25GbE (mlx5_0, eno12399)
NIC (experiment)	Mellanox ConnectX-5, 100Gbps	Mellanox ConnectX-5 Ex 100GbE (mlx5_2, ens1f0)
On-chip memory	256KB	64KB (actual allocatable; confirmed by ibv_exp_alloc_dm probe)
OS	CentOS 7.7.1908, Linux kernel 3.10.0	Ubuntu 20.04, MLNX_OFED 4.9-4.1.7.0

Component	Paper Configuration	Our Configuration
Threads	22 per node (176 total)	22 per node (44 total)

The experiment network interface on c6525-100g is ens1f0 (mlx5_2, 10.10.1.x, 100Gbps). The control network (mlx5_0, eno12399, 25Gbps) must not be used for RDMA traffic per CloudLab's acceptable use policy. [setup.sh (https://github.com/wilsonchang17/Sherman-CS6204/blob/main/setup.sh)](https://github.com/wilsonchang17/Sherman-CS6204/blob/main/setup.sh) auto-detects the correct interface by node type.

3. Setup Challenges and Solutions

3.1 MLNX_OFED Library Version Conflict

The most critical installation challenge was a version conflict between MLNX_OFED's libibverbs and Ubuntu's system packages. Sherman requires the ibv_exp_* experimental API (present only in MLNX_LIBS version 41mlnx1), but MLNX_OFED's default installation path installs UPSTREAM_LIBS (version 50mlnx1) which is missing this API.

Additionally, OFED's ibverbs-providers (50mlnx1) conflicts with Ubuntu's ibverbs-providers, causing apt to become completely broken after OFED installation.

Solution: Force-remove the conflicting packages and hold them to prevent apt from reinstalling them, then manually install libibverbs from MLNX_LIBS:

```
sudo dpkg --remove --force-depends ibverbs-providers python3-pyverbs
sudo apt-mark hold ibverbs-providers python3-pyverbs
sudo dpkg -i ${MLNX_LIBS}/libibverbs1_41mlnx1-*.deb
```

3.2 On-Chip Memory Size Detection

The paper reports using 256KB of NIC on-chip SRAM for the HOCL lock table. The original code has kLockChipMemSize = 256 * 1024. On c6525-100g nodes, ibv_exp_alloc_dm() can only allocate up to 64KB (compared to 128KB on r6525 nodes), causing a segfault if the hardcoded value exceeds the actual limit.

Solution: setup.sh now compiles and runs a small C probe that calls ibv_exp_alloc_dm() with decreasing sizes to find the actual allocatable limit, then patches kLockChipMemSize to that value automatically. This avoids node-type-specific hardcoding and handles any future hardware variation. The 64KB limit, combined with the open-source repo using uint64_t (8-byte) entries rather than the paper's 16-bit masked CAS, reduces lock table entries from 131,072 (paper: 256KB / 2B) to 8,192 (ours: 64KB / 8B), a 16x reduction that significantly affects lock contention behavior under skewed workloads.

The following C probe was used to determine the maximum allocatable device memory on mlx5_2:

```
#include <infiniband/verbs_exp.h>
size_t sizes[] = {256*1024, 128*1024, 64*1024, 32*1024, 0};
for (int i = 0; sizes[i]; i++) {
    struct ibv_exp_alloc_dm_attr attr = { .length = sizes[i] };
    struct ibv_exp_dm *dm = ibv_exp_alloc_dm(ctx, &attr);
```

```
    if (dm) { ibv_exp_free_dm(dm); printf("SUCCESS: %zu KB\n",
sizes[i]/1024); }
    else { printf("FAIL: %zu KB\n", sizes[i]/1024); }
}
```

Output on c6525-100g:

```
chenweiC@node1:~/Sherman-CS6204/build$ gcc /tmp/test_
dm.c -o /tmp/test_dm -libverbs && /tmp/test_dm
libibverbs: Warning: couldn't load driver 'mlx4': lib
mlx4-rdmav2.so: cannot open shared object file: No su
ch file or directory
FAIL:    256 KB
FAIL:    128 KB
SUCCESS: 64 KB
SUCCESS: 32 KB
```

3.3 RDMA Network Interface Selection

CloudLab nodes have multiple RDMA-capable NICs. On c6525-100g: mlx5_0/mlx5_1 map to the control network (25Gbps) and mlx5_2 maps to the experiment network (100Gbps, ens1f0, 10.10.1.x). Sherman's NIC selection in src/rdma/Resource.cpp selects the first device whose name ends in '0', which picks the control network by default.

Running RDMA workloads over the control network violates CloudLab's acceptable use policy. We patched Resource.cpp to select the device ending in '2' (mlx5_2, experiment network). The link rate was confirmed at 100 Gb/sec (4X EDR) using ibstat mlx5_2 and /sys/class/infiniband/mlx5_2/ports/1/rate. Traffic was verified using mlx5 port counters (/sys/class/infiniband/mlx5_*/ports/1/counters/port_xmit_data) since tcpdump cannot detect RDMA traffic due to kernel bypass.

Link rate verification on node0:

```
chenweiC@node0:~/Sherman-CS6204/results$ ibstat mlx5_
2 | grep -E "Rate|Speed"
                Rate: 100
chenweiC@node0:~/Sherman-CS6204/results$ cat /sys/cla
ss/infiniband/mlx5_2/ports/1/rate
100 Gb/sec (4X EDR)
```

3.4 Node ID Assignment with memcached

Sherman uses memcached to coordinate node ID assignment. It relies on memcached_increment on a key called serverNum. If this key does not exist before the benchmark starts, memcached_increment fails with NOT FOUND and the benchmark hangs.

Solution: Initialize the serverNum key to 0 before each benchmark run. Additionally, memcached must be bound to the experiment network IP (10.10.1.1) rather than localhost, and the systemd-managed memcached service must be disabled.

3.5 File Permission Issues

Running setup.sh with sudo leaves build artifacts owned by root, causing subsequent make and sed -i invocations to fail with Permission denied. Solution: Run sudo chown -R $USER

~/Sherman-CS6204/ before rebuilding. This also fixes perl -i failures, which need to create a temp file in the same directory.

3.6 Warmup Duration

The benchmark requires a warmup phase to populate the B+Tree with ~50 million key-value pairs before measurements begin. On c6525-100g with 2 nodes, warmup completes in approximately 13-14 seconds. We use a 150-second total timeout to allow warmup to complete reliably.

3.7 Stale memcached Keys Causing RDMA QP Failure

After each benchmark run, memcached retains stale QP connection keys. On the next run, Sherman reads these stale keys and attempts to build Queue Pairs using old connection parameters. The QP never transitions to Ready-to-Send (RTS) state because the remote QP no longer exists.

Root cause: Sherman does not flush memcached between runs. The serverNum reset is necessary but not sufficient. Solution: Run flush_all on memcached before every benchmark run, then reinitialize serverNum:

```
echo -e "flush_all\r\n" | nc 10.10.1.1 11211
echo -e "set serverNum 0 0 1\r\n0\r" | nc 10.10.1.1 11211
```

4. Results

All benchmarks were run on c6525-100g nodes (Utah cluster) using the experiment network interface mlx5_2 (ens1f0, 10.10.1.x, 100Gbps). The link rate was confirmed at 100 Gb/sec (4X EDR) using ibstat and /sys/class/infiniband/mlx5_2/ports/1/rate. Traffic was verified using mlx5 port counters: mlx5_2 transmitted approximately 40-70 GB per run while mlx5_0 (control network) showed zero traffic, confirming correct NIC selection.

Paper values are taken from Sherman's final configuration (+2-Level Ver column in Figures 10 and 11). Direct throughput comparison is not meaningful because our setup uses 2 nodes (44 threads) versus the paper's 16 nodes (176 threads). Per-thread throughput trends and latency values are the more meaningful comparison.

4.1 Uniform Workloads (Figure 11)

Workload: zipfian=0 (confirmed), 22 compute threads per node (44 total), 2 nodes. Warmup completed in approximately 13-14 seconds.

Workload	Read Ratio	Our Throughput (Mops)	Paper Throughput (Mops)	Our p50 (us)	Paper p50 (us)	Our p99 (us)	Paper p99 (us)
Write-only	0%	3.06	16.04	8.0	10.7	173	17.5
Write-intensive	50%	5.57	21.53	6.5	8.0	45	15

Workload	Read Ratio	Our Throughput (Mops)	Paper Throughput (Mops)	Our p50 (us)	Paper p50 (us)	Our p99 (us)	Paper p99 (us)
Read-intensive	95%	9.75	32.4	4.1	5.0	12.9	12.1

Uniform workloads show a clear trend: throughput scales from 3.06 Mops (write-only) to 9.75 Mops (read-intensive) as the read ratio increases. This is consistent with the paper's directional findings -- reads benefit more from the index cache and require fewer RDMA lock acquisitions than writes. Our absolute throughput is lower due to using 44 threads versus the paper's 176 threads (4x fewer). The p99 latency for write-heavy workloads (~173us) is higher than the paper's ~17.5us, attributable to CloudLab's experiment network switches lacking PFC/Priority Flow Control configuration for RoCE v2, causing occasional packet drops and retransmission.

4.2 Skewed Workloads (Figure 10)

Workload: zipfian=0.99 (verified), 22 compute threads per node (44 total), 2 nodes. Warmup completed in approximately 13-14 seconds.

Workload	Read Ratio	Our Throughput (Mops)	Paper Throughput (Mops)	Our p50 (us)	Paper p50 (us)	Our p99 (us)	Paper p99 (us)
Write-only	0%	3.06	4.14	8.4	9.6	174	1136
Write-intensive	50%	5.57	8.02	6.5	6.9	44	209
Read-intensive	95%	9.77	33.8	4.0	4.8	12.9	47

The skewed workload results are nearly identical to the uniform results. This is expected given our hardware constraint: the c6525-100g NIC can only allocate 64KB of device memory, providing 8,192 lock entries. With fewer lock slots, the HOCL lock table saturates under both uniform and skewed access patterns, reducing the differentiation that HOCL would provide under skewed access. The paper's 256KB lock table (131,072 entries, using 16-bit masked CAS) is better able to avoid contention under skewed workloads, explaining the dramatic p99 difference in write workloads (paper: 1136us skewed vs 17.5us uniform; ours: 174us vs 173us). This hardware limitation is an inherent constraint of the CloudLab environment and does not indicate a correctness issue with the reproduction.

Evidence of Lock Table Saturation

Three pieces of evidence confirm that lock table saturation explains the uniform/skewed convergence, and that the two workloads were genuinely run with different key distributions.

Figure A: Lock table entry count (include/Common.h)

```
chenweiC@node0:~/Sherman-CS6204$ grep -n "kLockChipMemSize\|kNumOfLock" ~/Sherman-CS6204/include/Common.h
88:constexpr uint64_t kLockChipMemSize = 64 * 1024;
93:constexpr uint64_t kNumOfLock = kLockChipMemSize / sizeof(uint64_t);
```

*kLockChipMemSize = 64 KB; kNumOfLock = 64*1024 / sizeof(uint64_t) = 8,192 entries. The open-source repo uses uint64_t (8-byte) lock entries rather than the paper's 16-bit masked CAS, so even with 256 KB the entry count would be 32,768 vs. the paper's 131,072. The combined effect of smaller on-chip memory (64 KB vs. 256 KB) and larger entry size (8 B vs. 2 B) reduces the lock table to 8,192 entries, 16x fewer*

than the paper, increasing hash collision rate proportionally and making uniform and skewed contention profiles converge.

Figure B: Uniform vs. skewed throughput and latency (write-only workload)

```
chenweiC@node0:~/Sherman-CS6204$ grep "cluster throughput" ~/Sherman-CS6204/results/result_uniform_writeon
ly.txt | tail -3
cluster throughput 3.051
cluster throughput 3.062
cluster throughput 3.061
chenweiC@node0:~/Sherman-CS6204$ grep "cluster throughput" ~/Sherman-CS6204/results/result_skewed_writeonl
y.txt | tail -3
cluster throughput 3.067
cluster throughput 3.064
cluster throughput 3.064
chenweiC@node0:~/Sherman-CS6204$ grep "p50\|p99" ~/Sherman-CS6204/results/result_uniform_writeonly.txt | t
ail -3
p50 8.000000    p90 12.400000   p95 29.800000   p99 173.400000       p999 239.000000
p50 8.000000    p90 12.400000   p95 29.800000   p99 173.400000       p999 239.000000
p50 8.000000    p90 12.400000   p95 29.70
chenweiC@node0:~/Sherman-CS6204$ grep "p50\|p99" ~/Sherman-CS6204/results/result_skewed_writeonly.txt | ta
il -3
p50 8.400000    p90 12.500000   p95 28.900000   p99 173.800000       p999 240.600000
p50 8.400000    p90 12.500000   p95 28.800000   p99 173.800000       p999 240.500000
p50 8.400000    p90 12.500000   p95 28.800000   p99 173.800000       p999 240.500000
```

Uniform write-only: cluster throughput ~3.06 Mops, p99 173.4 us. Skewed write-only: cluster throughput ~3.06 Mops, p99 173.8 us. The difference is under 0.3%, confirming that both workloads produce virtually identical lock contention profiles. In the paper, the same comparison yields a 65x p99 difference (17.5 us uniform vs. 1136 us skewed).

Figure C: Git history confirming separate binaries for uniform and skewed runs

```
chenweiC@node0:~/Sherman-CS6204$ git log --oneline --format="%h %ai %s" -- test/benchmark.cpp
bb57f76 2026-02-15 11:40:21 -0700 add results
a75a93b 2022-11-11 13:27:32 +0800 fix
2e96667 2022-11-04 12:29:28 +0800 fix
300dc9f 2022-10-31 15:15:03 +0800 fix data-race in index cache
fb55310 2022-10-31 14:05:40 +0800 clear
c5ee9d8 2022-02-16 20:39:02 +0800 wq
chenweiC@node0:~/Sherman-CS6204$ git show bb57f76 -- test/benchmark.cpp | grep "zipfan"
-double zipfan = 0;
+double zipfan = 0.99;
```

Commit bb57f76 (2026-02-15) changed zipfian from 0 to 0.99 in test/benchmark.cpp. Uniform results were collected before this commit (zipfian=0); skewed results were collected after (zipfian=0.99). This rules out the possibility that both result sets were produced by the same binary.

5. Conclusion

This report documents a successful reproduction of Sherman's benchmark infrastructure on CloudLab c6525-100g nodes, despite several hardware and software compatibility challenges. The core RDMA B+Tree functionality was validated on the experiment network interface (mlx5_2 / ens1f0, 100 Gbps) using mlx5 port counters.

All six workload configurations were completed. Under uniform workloads, throughput increased from 3.06 Mops in the write-only setting to 9.75 Mops in the read-intensive setting, consistent with the paper's qualitative trend. However, we found that Sherman's skewed-workload advantage is highly sensitive to hardware-specific NIC on-chip memory capacity. Under our CloudLab configuration, this advantage disappeared. In our environment, the effective NIC on-chip memory was limited to 64 KB, which restricted the HOCL lock table to 8,192 entries and caused skewed workloads to behave similarly to uniform workloads.

This result suggests that Sherman's reported skewed-workload benefit depends critically on the amount of available NIC on-chip memory and may not generalize to reduced-resource CloudLab settings.

6. Reproduction Artifacts

Artifact	Description	Location	Notes
setup.sh	Automated node configuration script	github.com/wilsonchang17/Sherman-CS6204	Works on r6525 and c6525-100g; auto-detects NIC on-chip memory size
RUNBOOK.md	Step-by-step experiment execution guide	github.com/wilsonchang17/Sherman-CS6204	Includes flush_all requirement and node-type-specific interface names
Patches	gidIndex, kLockChipMemSize, NIC selection	Applied by setup.sh automatically	kLockChipMemSize auto-detected by ibv_exp_alloc_dm probe
Results	Raw benchmark output files	result_uniform_writeonly.txt result_uniform_writeintensive.txt result_uniform_readintensive.txt result_skewed_writeonly.txt result_skewed_writeintensive.txt result_skewed_readintensive.txt	Collected on mlx5_2 (experiment network, ens1f0, 100Gbps)

All result files are available at: https://github.com/wilsonchang17/Sherman-CS6204/tree/main/results